PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference

Yufeng Gu*
University of Michigan
Ann Arbor, USA
yufenggu@umich.edu

Alireza Khadem*
University of Michigan
Ann Arbor, USA
arkhadem@umich.edu

Sumanth Umesh
University of Michigan
Ann Arbor, USA
sumanthu@umich.edu

Ning Liang
University of Michigan
Ann Arbor, USA
nliang@umich.edu

Xavier Servot
ETH Zürich
Zürich, Switzerland
xservot@student.ethz.ch

Onur Mutlu
ETH Zürich
Zürich, Switzerland
omutlu@gmail.com

Ravi Iyer†
Google
Mountain View, USA
raviiyer20@gmail.com

Reetuparna Das
University of Michigan
Ann Arbor, USA
reetudas@umich.edu

arXiv:2502.07578v3 [cs.AR] 3 May 2025

Abstract

Large Language Model (LLM) inference uses an autoregressive manner to generate one token at a time, which exhibits notably lower operational intensity compared to earlier Machine Learning (ML) models such as encoder-only transformers and Convolutional Neural Networks. At the same time, LLMs possess large parameter sizes and use key-value caches to store context information. Modern LLMs support context windows with up to 1 million tokens to generate versatile text, audio, and video content. A large key-value cache unique to each prompt requires a large memory capacity, limiting the inference batch size. Both low operational intensity and limited batch size necessitate a high memory bandwidth. However, contemporary hardware systems for ML model deployment, such as GPUs and TPUs, are primarily optimized for compute throughput. This mismatch challenges the efficient deployment of advanced LLMs and makes users pay for expensive compute resources that are poorly utilized for the memory-bound LLM inference tasks.

We propose CENT, a CXL-ENabled GPU-Free sysTem for LLM inference, which harnesses CXL memory expansion capabilities to accommodate substantial LLM sizes, and utilizes near-bank processing units to deliver high memory bandwidth, eliminating the need for expensive GPUs. CENT exploits a scalable CXL network to support peer-to-peer and collective communication primitives across CXL devices. We implement various parallelism strategies to distribute LLMs across these devices. Compared to GPU baselines with maximum supported batch sizes and similar average power, CENT achieves 2.3× higher throughput and consumes 2.9× less energy. CENT enhances the Total Cost of Ownership (TCO), generating 5.2× more tokens per dollar than GPUs.

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; **Neural networks**.

Keywords: Computer Architecture, Processing-In-Memory, Compute Express Link, Generative Artificial Intelligence, Large Language Models.

1 Introduction

Generative Artificial Intelligence (GenAI) has become pivotal in transforming a myriad of sectors. In the realm of content creation, Large Language Models (LLMs) [4, 29, 84, 106] provide assistance in writing, summarizing, and translating across diverse languages, revolutionizing the way textual content is produced. LLMs are reshaping various fields in daily life, such as generating creative arts [7, 83], customer services through chatbots, generating code and debugging assistance in software development [47]. However, harnessing the power of LLMs presents substantial economic challenges, underlined by their significant resource requirements. A business cost model indicates that running ChatGPT inference tasks requires ~3617 HGX A100 [75] servers and costs ~$694,444 per day [19]. Therefore, efficient and cost-effective

*Yufeng Gu and Alireza Khadem contributed equally to this research
†This research was done while the author was at Intel Corporation

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
ASPLOS '25, Rotterdam, Netherlands
© 2025 This is the author's version of the work. It is posted here by permission of ACM for your personal use. Not for redistribution. The definitive version was published in Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)
ACM ISBN 979-8-4007-1079-7/25/03
https://doi.org/10.1145/3676641.3716267

server farms play a critical role in the broader adoption and practical application of LLMs.

Decoder-only LLMs have witnessed exponentially larger parameter sizes. At the same time, LLMs use key-value (KV) caches to store context information, and modern LLMs support context windows from 128K to 1M to generate versatile texts, audios, and videos [29, 81]. Both the model parameters and KV caches require a large memory capacity. To meet this demand, advanced GPU stations feature multiple GPUs. However, the computational resources of multi-GPU systems are often underutilized in LLM inference tasks. Unlike earlier ML models, LLMs exhibit lower operational intensity characteristics, necessitating high memory bandwidth, primarily due to the sequential token generation and the lack of inherent parameter reuse. Although batching strategies could mitigate this issue, KV caches specific to each user require large memory capacity, limiting the feasibility of high batch sizes. Hence, the expensive compute throughput of GPUs and custom ML accelerators is significantly under-utilized for LLM inference because of the limited external memory bandwidth. As a result, *users pay for expensive computing resources for memory-bound LLM inference tasks.*

The high cost and low compute utilization of GPU systems motivate an alternative solution for LLM inference tasks. Processing-In-Memory (PIM) architectures [13, 14, 26, 40, 41, 50, 52, 55–58, 60, 68, 96, 118] place processing units (PU) adjacent to DRAM banks within memory chips, facilitating a significantly higher internal bandwidth. However, near-bank PUs, fabricated in the DRAM process, impose a high area overhead that reduces the memory density. A lower memory density is especially detrimental to LLMs with large memory requirements. On the other hand, Processing-Near-Memory (PNM) architectures [24, 27, 28, 46, 48, 51, 65, 72, 80, 87, 88] employ compute units near memory chips, *e.g.,* in memory controllers. PNM units are manufactured using CMOS process, offering more area-efficient compute capability at the cost of lower memory bandwidth compared to PIM.

PNM:
- per memory controller
- CMOS logic ⇒ fast

To address these challenges, CENT exploits Compute eXpress Link (CXL) [63] based memory expansion to provide the requisite memory capacity for LLMs. CENT establishes a practical CXL network to interconnect CXL devices. Each CXL device consists of 16 memory chips, with each chip containing two GDDR6-PIM channels, and compute units near these memory chips (PNM). This hierarchical PIM-PNM design supports the entire transformer block computation, eliminating the need for expensive GPUs.

CENT uses a CXL switch to connect multiple CXL devices, that are driven by a host CPU. The *inter-device communication* is enabled by CXL transactions [63]. The *intra-device communication* between PIM chips and PNM units is supported through a Shared Buffer. Using these protocols, CENT provides peer-to-peer and collective communication primitives such as *send/receive*, *broadcast*, *multicast* and *gather*. These primitives enable various parallelism strategies, efficiently distributing LLMs across CXL devices. In *Pipeline Parallel (PP)* [38] mapping, we assign each transformer block to multiple memory channels within a single CXL device, facilitating the concurrent processing of multiple prompts on different pipeline stages. PP prioritizes inference throughput to accommodate a large user base. In *Tensor Parallel (TP)* [100, 115] mapping, we distribute a transformer block across all CXL devices. TP focuses on reducing latency for real-time applications, providing smooth user experiences [25]. We also explore hybrid TP-PP mappings to strike a balance between the latency and throughput.

shared buffer: device 단위로 있다 보면 됨

(일부 애플리케이션 전용

PP: 각 transformer block을 memory (channel) group에 할당 (intra-device로는 병렬)
↳ 7개에 할당 함

TP: 32개의 device를 모두 사용 (inter-device communication 필요)
↳ Latency에 유리함

Within a CXL device, we introduce the detailed mapping of a transformer block onto the hierachical PIM-PNM architecture. In PIM chips, near-bank PUs incorporate Multiply-Accumulate (MAC) units, which support more than 99% of the arithmetic operations within a transformer block. The PNM units are composed of accelerators and RISC-V cores to perform other special and complex operations, such as Softmax, square root, and division. The integration of RISC-V cores allows for the flexible support of a wide range of LLMs.

In summary, this paper makes the following contributions:

- We propose CENT, a GPU-free system that uses CXL memory expansion to accommodate the considerable memory capacity requirements of LLMs. We design a hierarchical PIM-PNM architecture to support the entire transformer block computation, eliminating the need for expensive GPUs.
- We introduce a scalable CXL network to support collective and peer-to-peer communication primitives. We describe the mapping of LLM parallelization strategies across CXL devices based on the CXL communication primitives.
- We evaluate CENT on Llama2 [106] models. Compared to state-of-the-art GPUs with maximum supported batch sizes and similar average power, CENT achieves 2.3× higher throughput and consumes 2.9× less energy. CENT exhibits a lower Total Cost of Ownership (TCO), generating 5.2× more tokens per dollar than GPUs[1].

CENT is evaluated on Llama2 70B with a context length of up to 32K, but it can show higher benefits for larger model sizes and extended context lengths. As model sizes scale up, such as Grok 314B [111], Llama3 405B [18], and DeepSeek-V3 671B [64], inference serving demands significantly more hardware resources. In such cases, CENT offers greater cost-efficiency compared to GPUs.

decode attn 해당

In reasoning tasks [33, 82] and video generation [29, 83, 91], where context length can range from tens of thousands to 1 million tokens, CENT achieves higher throughput speedup due to its high memory bandwidth, which enhances memory-bound attention computations. Notably, GPUs can still benefit from long-text and video understanding tasks, as the

[1]Open-source CENT simulator https://github.com/Yufeng98/CENT/

prefill stage exhibits high operational intensity. In these scenarios, prefill and decoding processes can be disaggregated between GPUs and CENT, respectively [90, 116].

2 Motivation

The exponential growth of LLM parameters requires multi-GPU systems to accommodate the requisite memory capacity. However, LLMs exhibit limited operational intensity, making them memory-bound and resulting in suboptimal GPU utilization. Consequently, LLM service providers are paying significant costs for substantial computational throughput of multiple GPUs, which remains largely under-utilized.

LLM 하려면 memory 많이 필요함

High Memory Capacity Requirement. LLM parameter size has witnessed an exponential increase from Billion to Trillion magnitudes, far surpassing previous Machine Learning (ML) models. In addition, the context windows that modern LLMs support range from 128K to 1M [29, 81], enabling them to understand and generate longer contents. The long context window results in large KV caches, requiring substantial memory capacity. These KV caches are unique to each user, further limiting the ability to scale up the inference batch size due to the memory capacity requirement.

데이터 prefill / decode

Low Operational Intensity. LLM inference has two stages: (a) The *prefill* stage concurrently encodes input tokens within a prompt using matrix-matrix multiply (GEMM) operations. (b) The *decoding* stage decodes output tokens sequentially with matrix-vector multiply (GEMV) operations. The operational intensity of GEMV is substantially lower than GEMM. To mitigate this, several techniques are applied. Batching strategies combine GEMV operations across multiple queries of a batch into GEMM operations. This technique improves the operational intensity non-linearly because attention calculations rely on unique KV caches of each prompt. Grouped-query attention [3] merges multiple GEMV operations into narrow GEMM, but its operational intensity still remains less than the GPU capabilities.

GPU Performance Characterization. We use vLLM [54], the state-of-the-art inference serving framework, to study the effect of batch size and context length on 4 Nvidia A100 80GB GPUs running the Llama2-70B model [12, 106]. Figure 1 shows that inference throughput improves with larger batch sizes but reaches a plateau once the memory requirement exceeds the GPU memory size. As context length increases, inference throughput saturates with even smaller batch sizes, from batch=128 at 4K context length to batch=8 at 32K context length. Moreover, Figure 2(a) shows that LLM inference query latency increases with larger batch sizes and longer contexts, violating a realistic query latency Service Level Agreements (SLA) constraint [6].

Figure 1. Llama2-70B [12, 106] inference throughput and memory requirement on 4 A100 80GB GPUs.

Figure 2(b) compares the GPU compute utilization of an LLM (Llama2-70B [12, 106]) with an encoder-only transformer model (BERT [15]) and a Convolutional Neural Network (ResNet-152 [35]). BERT and ResNet-152 models predominantly consist of GEMM operations with high operational intensity, effectively utilizing GPU compute throughput. Conversely, LLama2-70B exhibits limited operational intensity, resulting in a mere 21% utilization of the available GPU compute throughput. Finally, decoding an output token in the decoding stage takes 3.4× longer than encoding a prompt token in the prefill stage due to the significant lower operational intensity of GEMV operations.

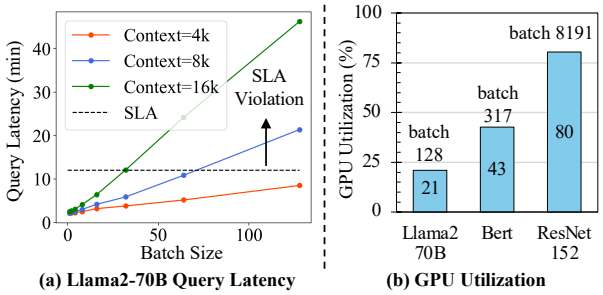


Figure 2. (a) Llama2-70B inference query latency increases with larger batches on 4 A100 80GB GPUs, Prompt size=512, Decoding size=3584. (b) GPU compute utilization, measured by Nvidia Nsight Compute profiler on 4 GPUs for Llama2-70B and 1 GPU for the other two models.

PIM Provides Higher Memory Bandwidth. Table 1 compares various manufactured industrial PIM prototypes and GPU. PIM enables the compute units to utilize the internal memory bandwidth, which significantly exceeds the external memory bandwidth of high-end GPUs with high bandwidth memories (HBM). For example, GDDR6-based AiM [56, 60] achieves 16 TB/s internal memory bandwidth compared to 2 TB/s external bandwidth of an A100 GPU with five HBM2E memory stacks. This large internal bandwidth coupled with a lower operational intensity makes PIM architectures a suitable alternative for expensive GPUs to perform LLM inference tasks.

Table 1. Hardware System Comparison

Type	PIM			GPU
Name	UPMEM	AiM	FIMDRAM	A100
Mem. Units	8 DIMMs	32 channels	5 stacks	5 stacks
Ex. BW (TB/s)	0.15	1	1.5	2
In. BW (TB/s)	1	16	12.3	-
Capacity (GB)	64	16	30	80
TFLOPS	0.5 TOPS[2]	16	6.2	312
Ops/Byte	0.5	1	0.5	156
Mem. Density	25%-50%	75%	75%	-

Low Memory Density of PIM. PIM suffers from a lower memory density due to the near-bank processing units that are fabricated in the DRAM process. For instance, DDR4-based UPMEM R-DIMM and GDDR6-based AiM reduce the memory capacity to 25%–50% and 75% compared to conventional DDR4 R-DIMMs and GDDR6 channels, respectively [13, 56]. An HBM2-based FIMDRAM cube consists of 4 PIM-enabled DRAM dies with 50% memory density and 4 conventional dies, lowering the memory capacity by 25% on average [57]. Given the lower memory density of PIM technologies and the substantial memory demands of LLMs, leveraging PIM as a scalable solution for LLMs presents significant challenges.

area overhead

Scalable Network of PIM. Scaling the memory capacity of PIM-enabled memories requires a scalable interconnect, efficient collective communication primitives, and parallelization strategies to optimally map LLMs to PIM devices. We utilize CXL 3.0 [63] as a low-latency interconnect protocol, built on top of the PCIe physical layer. CXL 3.0 supports inter-device communication through a CXL switch. Compared to network-based RDMA, CXL.mem offers ~8× lower latency [31]. The CXL 3.0 protocol can support up to 4,096 nodes, exhibiting better scalability than NVLink [76]. NVLink provides higher bandwidth (at a higher cost), which is critical for LLM training. However, we show that the lower bandwidth of CXL is not a bottleneck for LLM inference due to the limited volume of data transfers in various parallelization strategies.

CXL 4G

To distribute the LLMs, we detail the mapping of the transformer blocks to the CXL devices based on the Pipeline Parallel (PP) [100, 115] and Tensor Parallel (TP) [38] strategies. For PP, we provide peer-to-peer *send* and *receive* primitives for the transmission of the embedding vector between the pipeline stages across CXL devices. For TP, we implement *gather* and *broadcast* collective communication primitives to transfer partial results. To balance the throughput and latency of the network, we study the hybrid TP-PP parallelization strategy using the *multicast* primitive.

Hierarchical PIM-PNM Architecture. In addition to GEMV, a transformer block contains different layers, such RMSNorm [113], Rotary Embedding [102], and SiLU [22]. For the end-to-end execution of transformer blocks as an alternative to costly GPUs, there are two options: (a) Perform all operations near-bank using a general-purpose PU similar to UPMEM [13] architecture. (b) Perform MAC operations of GEMVs in domain-specific near-bank PUs similar to AiM [56], and assign other operations to the PNM units, shared by multiple PIM chips. We use the second approach and propose a hierarchical PIM-PNM solution because of two primary reasons: First, a general-purpose near-bank PU incurs more overhead on memory density and yields lower compute throughput compared to domain-specific alternatives. Second, MAC operations constitute over 99% of arithmetic operations within a transformer block, rendering general-purpose near-bank PUs over-provisioned for other infrequent arithmetic operations.

[2]UPMEM supports only integer precision, so unit is TOPs.

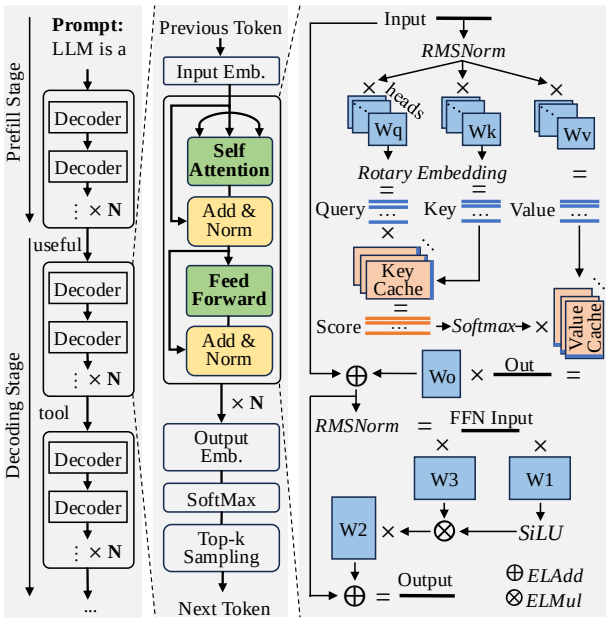


Figure 3. (a) Prefill stage encodes prompt tokens in parallel. Decoding stage generates output tokens sequentially. (b) LLM contains N× decoder transformer blocks. (c) Llama2 model architecture.

3 Background

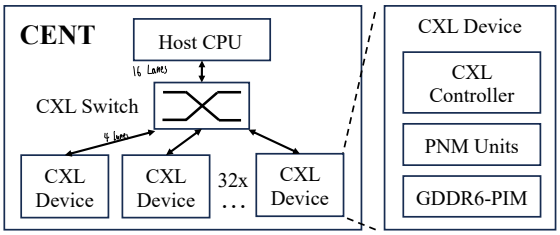
Figure 3(a) shows that a decoder-only LLM initially processes a user prompt in the "prefill" stage and subsequently generates tokens sequentially during the "decoding" stage. Both stages contain an input embedding layer, multiple decoder transformer blocks, an output embedding layer, and a sampling layer. Figure 3(b) demonstrates that the decoder transformer blocks consist of a self attention and a feed-forward network (FFN) layer, each paired with residual connection and normalization layers.

Figure 3(c) demonstrates the Llama2 [106] model architecture as a representative LLM. In the self-attention layer, query, key and value vectors are generated by multiplying

input vector to corresponding weight matrices. These matrices are segmented into multiple heads, representing different semantic dimensions. The query and key vectors go though Rotary Positional Embedding (RoPE) to encode the relative positional information [102]. Within each head, the generated key and value vectors are appended to their caches. The query vector is multiplied by the key cache to produce a score vector. After the Softmax operation, the score vector is multiplied by the value cache to yield the output vector. The output vectors from all heads are concatenated and multiplied by output weight matrix, resulting in a vector that undergoes residual connection and Root Mean Square layer Normalization (RMSNorm) [113]. The residual connection adds up the input and output vectors of a layer to avoid vanishing gradient [35]. The FFN layer begins with two parallel fully connections, followed by a Sigmoid Linear Unit (SiLU), and ends with another fully connection.

4 CENT Architecture

Figure 4 presents the CENT architecture, where a CXL switch interconnects 32 CXL devices, driven by a host CPU. Each CXL device integrates a CXL controller, PNM units, and 16 memory chips, each equipped with two GDDR6-PIM channels (hereafter referred to as PIM channels). We introduce a CXL-based network architecture, a hierarchical PIM-PNM design and the CENT ISA in this section.



16 Mem chips (controllers)
2 PIM Channel per chips
↳ 32 PIM Channel

Figure 4. CENT Architecture.

4.1 CXL-based Network Architecture

CENT integrates the CXL 3.0 protocol, using the PCIe 6.0 physical interface. The CXL switch is connected to the host machine with x16 lanes, whereas each CXL device is connected to the switch through x4 lanes. The switch supports the communication between the host and CXL devices, and peer-to-peer communication between CXL devices.

Inter-Device Communication. Figure 5 shows the architecture of a CXL device. Communication between CXL devices involves the Shared Buffer and is orchestrated by the inter-device communication controller in conjunction with the CXL port. We introduce a *broadcast* primitive, allowing one CXL device to write data to multiple devices through a single request. The standard CXL.mem protocol lacks this support. We implement it by using one of the reserved header codes within the Header slot (H-slot) of the Port Based Routing (PBR) flit. The H-slot is decoded by the switch for routing. Upon identifying a flit encoded with this reserved H-slot code, the switch interprets it as a broadcast request and forwards the flit to designated CXL devices. We also modified the CXL port to (1) incorporate a device ID mask within the header slot of the broadcast message, and (2) expect write acknowledgements from all destination devices.

device간 communication에는 shared buffer를 거치며, inter-device com controller와 CXL 포트가 필요

CXL switch에 broadcast 기능 추가

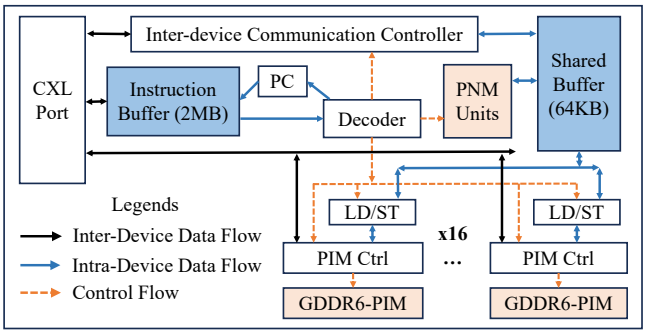


Figure 5. CXL Device Architecture.

Inter-device communication is supported by SEND_CXL, RECV_CXL and BCAST_CXL instructions. The non-blocking SEND_CXL specifies the device ID (DVid) and the Shared Buffer address in source and destination devices. Conversely, RECV_CXL operates in a blocking manner and does *not* specify a device ID. A pair of send and receive instructions constitutes a CXL write transaction. BCAST_CXL is also non-blocking and uses an 8-bit DVcount parameter to specify the number of subsequent CXL devices to which the data is broadcast. The *multicast* primitive is supported in a similar manner. To accomplish *gather*, the receiving device executes multiple CXL_RECV instructions, while each sender executes one SEND_CXL instruction. Note that the receive instruction omits any device ID specification, thereby rendering the order of incoming CXL flits inconsequential.

SEND_CXL
└ non-blocking / DVid + addr 제공
RECV_CXL - 내 a 주소 (내 buffer 영역 지정하기)
└ blocking / 내 buffer addr 제공
↳ 모든 도착까지 대기
BCAST_CXL
└ non-blocking / DV count 제공 → 내 DVid 인접 DV에 전송 내용
GATHER

CXL Port is depicted in Figure 6. CXL nodes are classified into three categories: Host (H), representing the host machine; Local (L), the CXL device we are considering; and Remote (R), referring to other CXL devices interconnected via the switch. CXL port is equipped with virtual channels. Requests from the host and remote nodes are unpacked onto the Rx H2L and R2L queues, and responses to the host and remote nodes are allocated to the Tx L2H and L2R queues.

PIM Channel 1개가 아닌 PIM device 단위

Transactions comprise a request and a response. On the transmit (Tx) datapath, the CXL port packs requests into flits, which are unpacked on the receive (Rx) datapath by the destination device. The CXL port supports 2 types of transactions: read transactions, initiated with a *Request* (Req) and concluded with *Data with Response* (DRS); and write transactions that begin with a *Request with Data* (RWD) and finish with *No Data Response* (NDR) acknowledgment.

4.2 Hierarchical PIM-PNM Architecture

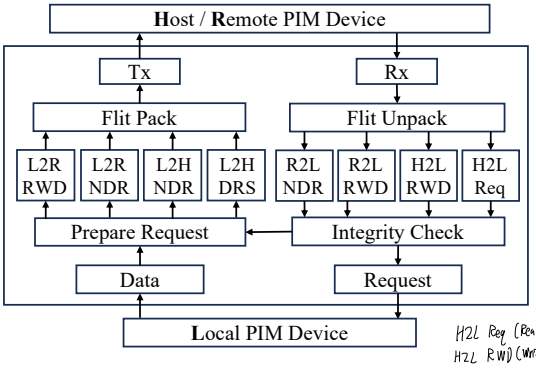


Figure 6. CXL Port Architecture.

In Figure 5, CENT instructions are transmitted from the host to a 2MB instruction buffer in each device. These instructions are further distributed to PIM channels and PNM units. Standard read/write transactions are dispatched to PIM controllers similar to non-PIM memory modules. CENT arithmetic instructions are decoded into micro-ops and subsequently directed to PIM controllers and PNM units.

GDDR6-PIM Channel. The CXL device integrates 16 PIM controllers, each managing two PIM channels. These controllers receive micro-ops from the decoder and convert them into DRAM commands. Figure 7(a) shows that the PIM channel consists of a 2KB Global Buffer shared by four bank groups. The bank group contains four banks. Each bank has a 32MB memory capacity coupled with a near-bank PU.

Within the PU is a 16 MAC reduction tree, operating on Bfloat16 (BF16) data elements. Each multiplier receives 16-bit data directly from its associated local bank, in addition to another 16-bit data from either the Global Buffer or its neighboring bank (such as Bank 0 and Bank 1). The Global Buffer is capable of broadcasting 256-bit data to all PUs concurrently. 32 accumulation registers are incorporated to hold the MAC results in the PU and are designated by the CENT ISA. The activation function (AF) leverages lookup tables stored within the DRAM bank and linear interpolation.

The PU operates at 1GHz, equivalent to t_{CCDS} ($2t_{CK}$) of the PIM bank, yielding a compute throughput of 32 GFLOPS. PIM channels are optimized to allow 16 near-bank PUs to operate in parallel. To facilitate this, the PIM controller issues an activate-all-banks ACTab command, followed by PIM commands such as MACab and concludes with a precharge-all-banks PREab command. The ACTab command is enabled by the reservoir capacitors introduced in AiM [56, 60], and PREab is already supported by the GDDR6 DRAM [97].

PNM Units. While near-bank PUs could efficiently support MAC operations, LLMs necessitate a broader set of operations beyond MACs. To address this, the CXL device incorporates the following PNM units, as shown in Figure 7(b): (1) *32 Accumulators*: each retrieves two values from the Shared Buffer as inputs and segments the 256-bit inputs into 16 groups for BF16 accumulations. (2) *32 Reduction Trees*: each fetches a single 256-bit value from the Shared Buffer, reducing 16 BF16 input elements to a single BF16 value. The result is stored into the first 16-bit element in a 256-bit Shared Buffer slot. (3) *32 Exponent Accelerators*, each accesses a 256-bit value from the Shared Buffer, dividing it into 16 lanes. In each lane, the exponent of a BF16 input element is calculated by a 10-order Taylor Series approximation. (4) *8 BOOM-2wide RISC-V cores* [10], facilitating the execution of less common operations (such as square root and inversion), and accommodating future improvements in LLMs. Each RISC-V core is equipped with a 64KB instruction buffer, which is initialized by the host through CXL write transactions.

Intra-Device Communication between PIM channels and PNM units is enabled through CENT data movement instructions and a 64KB Shared Buffer. The Shared Buffer is viewed by PIM channels as 256-bit registers. CENT facilitates data transfers between DRAM banks and the Shared Buffer by WR_SBK and RD_SBK instructions. These transfers are conducted by the load/store unit associated with each memory controller. Additionally, WR_ABK instruction segments a 256-bit register into 16 discrete BF16 values and concurrently stores them in the same row and column address of all 16 banks within a channel. Communication among banks in a PIM channel is mediated by the Global Buffer through COPY_BKGB and COPY_GBBK instructions. Similar to PIM channels, PNM units interface with the Shared Buffer at a 256-bit granularity and abstract it as a register file. The RISC-V core views the Shared Buffer as a byte-addressable memory and interacts with it through 16-bit loads and stores in a designated 64KB region of the memory space.

4.3 ISA Summary

Table 2 shows CENT arithmetic instructions. The CHmask parameter directs the PIM decoder to broadcast micro-ops to specified PIM channels. PIM decoder generates OPsize micro-ops from a single instruction, targeting subsequent Shared Buffer slots and DRAM column addresses. The Regid parameter identifies the specific accumulation register within the PU, while AFid determines the type of non-linear activation function. The RISCV instruction is designed to initiate the execution of RISC-V cores at the specific start program counter (PC) address.

Table 2. CENT Arithmetic Instructions

Instruction	Assembly
Near-Bank PUs	
MAC All Bank	MAC_ABK CHmask OPsize RO CO Regid
Element-wise Mult.	EW_MUL CHmask OPsize RO CO
Activation Function	AF CHmask AFid Regid
PNM Units	
Exponent	EXP OPsize Rd Rs
Reduction	RED OPsize Rd Rs
Accumulation	ACC OPsize Rd Rs
RISCV operation	RISCV OPsize PC Rd Rs

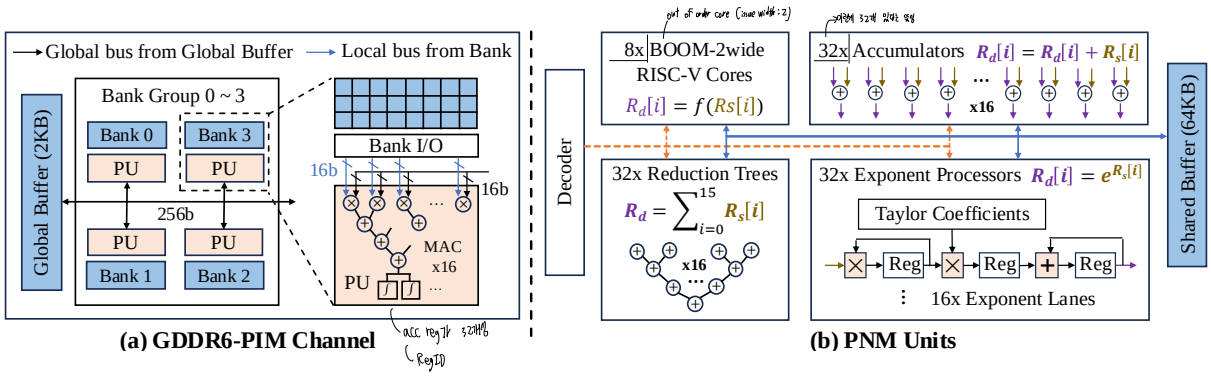


Figure 7. Hierarchical PIM-PNM Architecture

Table 3 summarizes CENT data movement instructions, specifying DRAM bank locations using channel (`CHid`), bank (`BK`), row (`RO`), and column (`CO`). The source and destination Shared Buffer addresses are specified by `Rd` and `Rs`.

Table 3. CENT Data Movement Instructions

Instruction	Assembly
CXL Device ↔ CXL Device	
Send	`SEND_CXL DVid Rs Rd`
Receive	`RECV_CXL`
Broadcast	`BCAST_CXL DVcount Rs Rd`
Shared Buffer ↔ DRAM Banks	
Write Single Bank	`WR_SBK CHid OPsize BK RO CO Rs`
Read Single Bank	`RD_SBK CHid OPsize BK RO CO Rd`
Write All Banks	`WR_ABK CHid RO CO Rs Regid`
Global Buffer ↔ DRAM Banks	
Copy Bank → Global Buffer	`COPY_BKGB CHmask OPsize RO CO`
Copy Global Buffer → Bank	`COPY_GBBK CHmask OPsize RO CO`
Shared Buffer ↔ PUs	
Write bias	`WR_BIAS CHmask Rs`
Read MAC register	`RD_MAC CHmask Rd Regid`
Shared Buffer → Global Buffer	
Write Global Buffer	`WR_GB CHmask OPsize CO Rs`

5 Model Mapping

The ever-increasing parameter size of the LLMs, coupled with the lower memory density of PIM, necessitates the distribution of the LLM inference on a scalable network of PIM modules. In this section, we introduce the mapping of various LLM parallelization strategies on CENT's CXL-based network architecture using the proposed collective and peer-to-peer communication primitives.

5.1 Pipeline-Parallel Mapping (PP)

Cloud providers serve a large user base, where inference throughput is crucial. To improve throughput, PP [38] assigns each transformer block to a pipeline stage. The individual queries in a batch are simultaneously processed in different stages of the pipeline. Figure 8 shows that we map multiple pipeline stages (*e.g.*, `T0-3`) to a CXL device (*e.g.*, `D0`). Each stage requires multiple PIM channels, depending on the memory requirements of the decoder block. To prevent excessive communication and keep the latency of pipeline stages identical, we avoid splitting a pipeline stage between the PIM channels of two CXL devices.

In each iteration, the output of each transformer block is transferred to the next pipeline stage. CENT performs this data transfer using intra-device communication for pipeline stages within the same CXL device, and using peer-to-peer *send* and *receive* primitives for those in different CXL devices. This CXL data transfer contains only an 8K embedding vector (16KB data) in Llama2-70B. The CXL transfer latency of PP is negligible compared to PIM and PNM latencies.

Note that CENT does not support batch processing within a single pipeline stage because of two primary reasons: First, batching requires a significantly larger Global Buffer and Shared Buffer (Section 4.2) to concurrently store the embedding vectors of multiple queries. Second, batching enhances the operational intensity and compute utilization (Section 2), while PP fully utilizes PIM compute resources. Therefore, applying batching on top of PP only increases the latency.

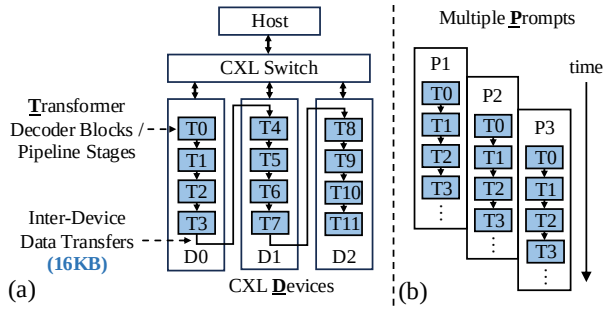


Figure 8. Pipeline parallelism: (a) Transformer decoder blocks are distributed across CXL devices and form the pipeline stages. Each block is mapped to multiple GDDR6-PIM channels. (b) Multiple prompts are executed in different stages of the pipeline.

5.2 Tensor-Parallel Mapping (TP)

Inference latency is critical in real-time applications to provide a smooth user experience [25]. To enhance the latency,

TP [100, 115] uses all compute resources to process decoder blocks one at a time. To implement TP, Figure 9(a) shows that CENT assigns each transformer decoder block across all CXL devices. Figure 9(b) illustrates the detailed mapping of a transformer block using TP. The infrequent residual connection and normalization layers are confined within a single master CXL device. Distributing the attention layer requires the frequent use of expensive *AllReduce* collective communication primitive, which significantly increases the CXL communication overhead [100]. Consequently, the attention layer is mapped to the master CXL device.

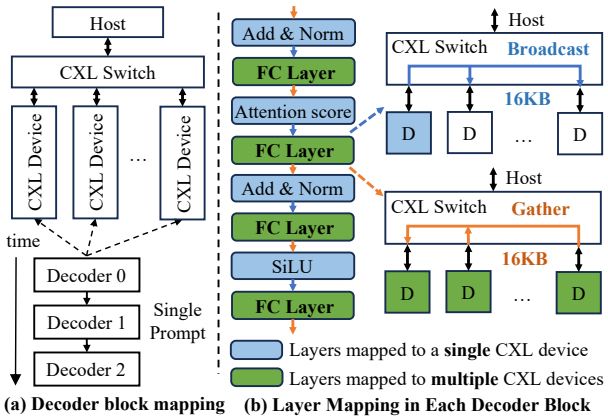


Figure 9. (a) Tensor parallelism: each transformer block is assigned to multiple CXL devices. Prompts are processed sequentially. (b) In a transformer block, fully connected layers are spread across CXL devices, while other operations are confined to a single device.

Prior to the execution of an FC layer, the embedding vector (16KB for Llama2-70B) is *broadcast* from the master CXL device to all devices via the CXL switch. This enables each device to locally perform GEMV on multiple rows of the weight matrix. Following the execution of an FC layer, partial result vectors are *gathered* to the master CXL device. This approach optimizes the execution of FC layers across multiple devices, while reducing the communication overhead of TP through the CXL switch to only 135KB data transfer for each transformer block of the Llama2-70B model.

5.3 Hybrid Tensor-Pipeline Parallel Mapping

The TP and PP mappings focus either on inference latency or throughput. However, balancing both can be crucial in real-world deployment scenarios when considering Quality of Service (QoS) requirements [6]. We explore a hybrid TP-PP strategy to achieve this balance, where each transformer decoder is allocated to multiple consecutive CXL devices. For example, among 32 devices, mapping each decoder to $32/4 = 8$ devices enables TP=8 and PP=4. The embedding vectors are *multicast* and *gathered* by the master CXL device of each pipeline stage. This configuration effectively reduces token decoding latency by utilizing compute resources from multiple CXL devices (TP), while also improving the throughput by processing multiple prompts in parallel (PP).

5.4 Transformer Block Mapping

CENT involves a fine-grained mapping of the transformer block onto CXL devices, PNM accelerators, and PIM channels. This technique permits the complete execution of a transformer block within the CXL device, thereby eliminating the necessity for any interaction with the host system. Figure 10(a) illustrates the operations within a Llama2 transformer block. Operations within the blue blocks are assigned to PIM channels, including GEMV in fully connected layers, vector dot product in RMSNorm, and element-wise multiplication in RMSNorm, SiLU, Softmax and Rotary Embedding, as detailed in Figure 10(b), (c), (d), and (e), respectively. On the other hand, model-specific operations marked in orange, such as square root, division, Softmax, and vector addition in residual connections, are handled by the PNM's RISC-V cores and accelerators. CENT supports *Grouped-Query Attention* [3] in Llama2-70B by unrolling GEMM to GEMV.

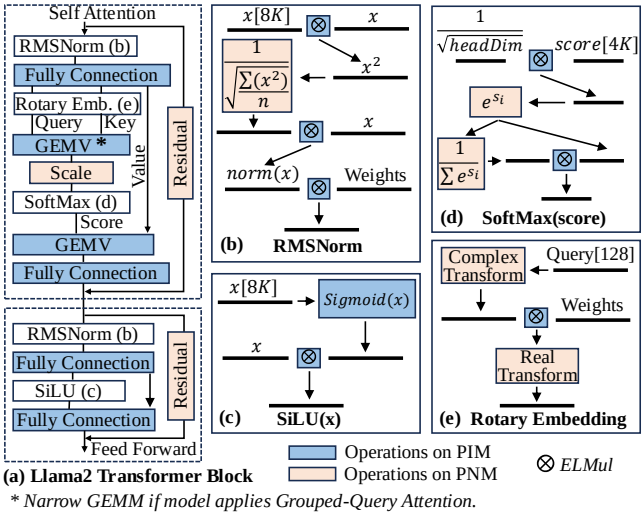


Figure 10. (a) Llama2-70B Transformer Block. Blue and orange operations are mapped to PIM and PNM PUs, respectively. (b)~(e) Operation mapping for RMSNorm, SiLU, SoftMax and Rotary embedding.

In Figure 10(d), the score dimension varies between 1 and $4k$, accommodating the 4K sequence length in this example. The embedding dimensions, as shown in Figure 10(b) and (c), are set to $8K$. The rotary embedding process, depicted in Figure 10(e), begins with the RISC-V PNM cores transforming an attention head of dimension 128 into 64 groups of the complex number representations (*e.g.*, $[a, b, c, d]$ to $[(a + jb), (c + jd)]$). The PIM PUs within memory chips then multiply complex values and pre-loaded weights. Finally, RISC-V PNM cores convert the computed results back to their real value representations.

CENT's PIM computations include three key operations. This paragraph explains the execution of each operation within a GDDR6-PIM channel. (a) *GEMV*: The matrix is partitioned along its rows and distributed across all 16 banks. The vector is transferred to the Global Buffer. `MAC_ABK` instructions then broadcast 256-bit vector segments from the Global Buffer to all near-bank PUs, retrieve 256-bit segments of the matrix rows from the banks, and perform MAC operations. (b) *Vector dot product*: In this operation, input vectors are stored in neighboring banks. `MAC_ABK` instructions retrieve 256-bit segments from these banks and perform MAC operations. Throughout this process, only one of the two neighboring near-bank PUs is utilized. (c) *Element-wise multiplication*: Before this operation, input vectors are stored in two banks within each bank group, which consists of four banks. `EW_MUL` instructions then retrieve 256-bit segments from these two banks, perform the multiplication, and store the results in another bank within the same bank group.

5.5 End-to-End Model Mapping

이건... 어떻게 한다?
WOW...

CENT supports the end-to-end query execution in LLM inference tasks. In the prefill stage, CENT processes tokens in the prompt one after another to fill out KV caches, using a similar approach to that in the decoding stage. Within each token, both input embeddings and transformer blocks are mapped to CXL devices using the mapping techniques introduced in Section 5.4. In the decoding stage, after a series of transformer blocks, the top-k sampling operations are executed on the host CPU.

5.6 Programming Model

Users can specify the CENT hardware configuration, including the number of PIM channels to utilize, and the number of pipeline stages. The tensor mapping strategy is determined by this configuration. CENT library provides Python APIs to allocate memory space and load model parameters according to the model mapping strategy. These APIs also support commonly used LLM operations, such as `GEMV`, `LayerNorm`, `RMSNorm`, `RoPE`, `SoftMax`, `GeLU`, `SiLU`, *etc.* CENT uses an in-house compiler to generate arithmetic and data movement instructions illustrated in Section 4.3.

```
1.  Vector = {shared_buffer_addr, vector_dim}
2.  Matrix = {num_row, row_addr, matrix_dim}
3.  def GEMV(Vector, Matrix, Hardware_config) {
4.      for each channel_index in Hardware_config->num_channels:
5.          WR_GB (Vector->shared_buffer_addr)
6.          for each row_index in Matrix->num_rows:
7.              WR_BIAS (channel_index)
8.              MAC_ABK (channel_index, row_addr + row_index)
9.              RD_MAC (channel_index)
10. }
```

Figure 11. Vector-matrix multiplication compilation

Figure 11 shows an example of compiling `GEMV` to CENT instructions. Initially, the operands are designated to particular memory spaces, *i.e.*, the vector operands in the Shared Buffer and the matrix operands in PIM channels (lines 1 and 2). CENT instructions are then generated based on input operands' dimensions and memory addresses. Subsequently, the vector is copied to the Global Buffers in the PIM channels with `WR_GB` instructions (line 5). This is followed by a sequence of operations for each matrix row within the near-bank PIM PUs. The `WR_BIAS` instruction sets up the accumulation registers (line 7). `MAC_ABK` performs the multiply-accumulate operations across all near-bank PUs in the PIM channel (line 8). Finally, `RD_MAC` retrieves the results from the accumulation registers (line 9).

6 Methodology

Table 4 lists the system configurations of CENT and our GPU baseline. The GPU system contains 4 NVIDIA A100 80GB GPUs equipped with the NVLink 3.0 interconnect. CENT has 32 CXL devices, resulting in a similar average power to the GPU system, as further explained in Section 7.2.

Table 4. Evaluated system configurations

System	CENT	GPU
Hardware	32 CXL devices	4 NVIDIA A100
Process	1Y nm (14-16nm)	7nm
Memory	512GB, GDDR6	320GB, HBM2E
Compute Throughput	512 TFLOPS (PIM)	1248 TFLOPS
	96 TFLOPS (PNM)	
Peak Bandwidth	512 TB/s (Internal)	8 TB/s (External)
3-Year Owned TCO	0.73$/hour	1.76$/hour
3-Year Rental TCO	1.05$/hour	5.45$/hour
GDDR6-PIM Timing Constraints	t_{RCDRD}=18ns, t_{RAS}=27ns, t_{CL}=25ns t_{RCDWR}=14ns, t_{CCDS}=1ns, t_{RP}=16ns	

We benchmark Llama2 7B, 13B, and 70B models [106]. Each evaluated query contains 512 tokens in the prefill stage and 3584 tokens in the decoding stage, adding up to a context length of 4K, *i.e.,* the maximum supported by the Llama2 models. For a fair comparison between CENT and the GPU baseline, we deploy these models using different configurations for different parameter sizes: 1, 2, and 4 GPUs, and 8, 20 and 32 CXL devices. We use vLLM [54], the state-of-the-art inference serving framework on GPUs with a batch size of 128, where the inference throughput saturates (Figure 1).

We generate CENT instruction traces for a single block and verify the correctness using a functional simulator. We modify Ramulator2 [67] to model a CXL device containing 32 GDDR6-PIM memory channels with timing constraints in Table 4. The inter-device communication through the CXL 3.0 protocol is modeled by an analytical model based on the CXL latency [61] and PCIe 6.0 bandwidth. To model a CXL switch supporting multicast, we use half of the bandwidth

and double the latency of the baseline switch. We use Intel Xeon Gold 6430L CPU [42] as the host machine in CENT.

We use Micron DRAM Power Calculator [69] to evaluate DRAM core power using current and voltage specifications of Samsung's 8Gb GDDR6 SGRAM C-die [97]. The MAC operation power is modeled assuming 3× more current than a typical gapless read [56]. We assume that each GDDR6 memory controller for two channels consumes 314.6 mW [108] and each BOOM RISC-V core consumes 250 mW [9]. We implement the RTL of the remaining components in the CXL controller and synthesize it using a TSMC 28nm technology library and the Synopsys Design Compiler [104]. We find the critical path delay as 1ns at 28nm and project the CXL controller clock frequency to be 2.0 GHz at 7nm [101].

We estimate the die area of CXL controller in two parts. First, we synthesize the custom logic in 28nm (See Table 5) and scale it down to 7nm [101]. Then, we add measurements of the memory controller, PCIe controller, and PHY from the NVIDIA GPU die shots [89, 110], which are also scaled down to 7nm. This results in an estimated area of 19.0mm^2 in 7nm.

Table 5. CXL Controller Custom Logic Area&Power in 28nm

Components		Area (mm^2)	Power (W)
SRAM	Instruction Buffer	3.33	0.61
	Shared Buffer	0.11	0.03
Logics	Accelerators	1.34	0.18
	RISC-V Cores	2.94	0.19
	Others	0.12	0.05
Total		**7.85**	**1.06**

Table 4 presents the 3-year Total Cost of Ownership (TCO) for both owned and rental hardware. (a) *Own TCO:* We model a local server by accounting for hardware and operational costs. (b) *Rental TCO:* The cost for host CPU in CENT and GPU are estimated based on the Microsoft Azure prices [1]. The CXL devices in CENT are evaluated using the owned TCO methodology, as there are no available references for rental costs. To calculate operational cost, we use \$0.139/KWh [79] and average power consumption. Hardware costs are listed in Table 6. While the lowest available price for A100 80GB is close to \$20,000, we instead use only \$10,000 by conservatively deducting 50% margin [20]. The PIM module cost is estimated as 10× the cost of standard DRAM modules [17, 107].

Table 6. Hardware Costs

System	Hardware	Cost (\$)
GPU	Xeon Gold 6430 CPU [43]	2,128
	4 NVIDIA A100 80GB GPU [20]	40,000
	Total Cost	**42,128**
CENT 32 devices	Xeon Gold 6430 CPU [43]	2,128
	512GB GDDR6-PIM [17, 107]	11,873
	32 CXL Controllers	381.3
	96-lane 48-port switch [21]	490
	Total Cost	**14,873**

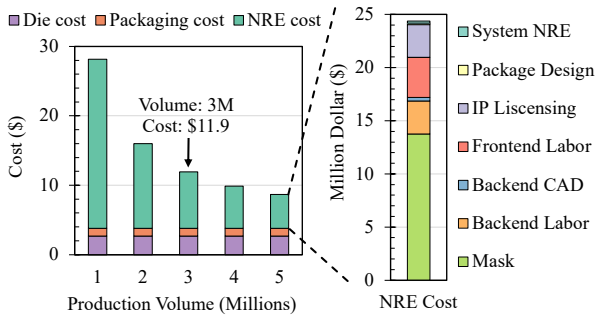


Figure 12. CXL Controller Cost Breakdown

Figure 12 illustrates the breakdown of CXL controller cost per CENT CXL device (Figure 5). The CXL controller costs are broken down into die, packaging and Non Recurring Engineering (NRE) cost components [49, 71]. Die cost is derived from the wafer cost, considering the CXL controller die area (19.0mm^2 in 7nm) and yield rate. A 300mm diameter 7nm wafer costs \$9,346 with a defect density of 0.0015 per mm^2 [71]. Cost of 2D packaging is assumed to be 29% of chip cost [59], while the 2.5D packaging cost is calculated based on interposer, die placement and substrate assembly [85]. NRE cost is influenced by chip production volumes, which we estimate at 3 million units based on the following assumptions. NVIDIA shipped 3.76M datacenter GPUs in 2023 [70]. We assume that 10% of datacenter GPUs (around 370K) are used for LLM inference. Since each GPU consumes ~8× more power compared to a CENT device (explained in Section 7.2), we project ~3M volume for CENT devices.

7 Results

7.1 CENT versus GPU Baseline

Figure 13 compares the performance of CENT and our GPU baseline under two scenarios: (a) *Latency Critical:* We use a batch of 1 query (CENT's tensor parallel mapping).

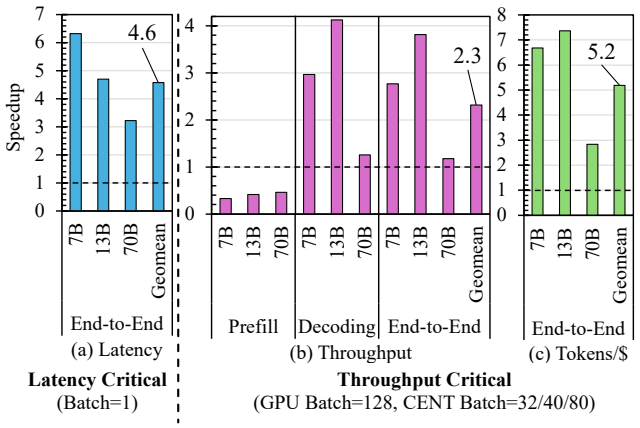


Figure 13. CENT speedup over GPU baselines.

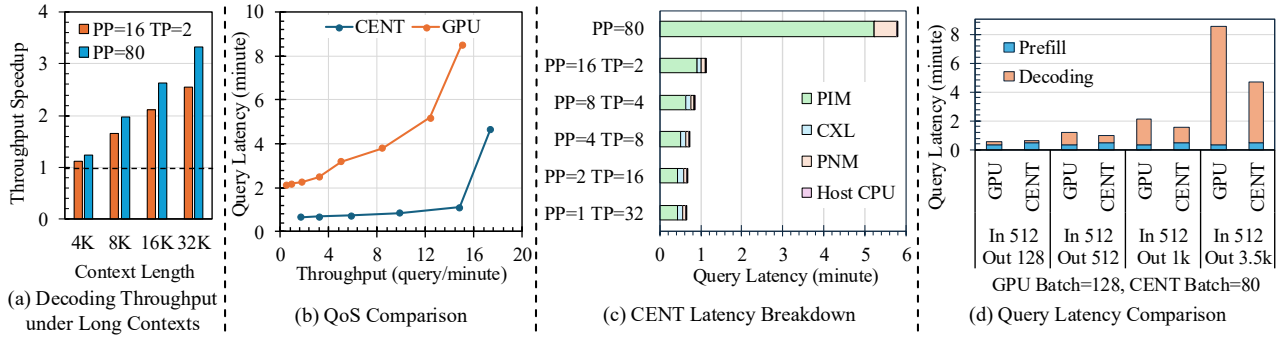


Figure 14. Analysis on Llama2-70B. (a) CENT achieves higher decoding throughputs with long context windows and 3584 decoding sizes (Section 6). 16K and 32K context scenarios with PP=80 configurations require the 16Gb GDDR6-PIM module, increasing CENT capacity to 1TB (2X of that used in main results). (b) QoS analysis: CENT provides lower query latency while achieving similar throughput as GPU. (c) CENT latency breakdown with different parallelism strategies. (d) Prefill (In) and decoding (Out) latency comparison with different In/Out sizes, at maximum supported batch size for both GPU and CENT.

In this case, CENT reduces the end-to-end latency by 4.6× compared to GPUs. This speedup is due to the higher internal memory bandwidth of PIM. (b) *Throughput Critical:* We use the maximum batch size of 128 for GPU experiments, as explained in Section 6. On the other hand, CENT utilizes pipeline parallelism to enable batches of 32/40/80 queries for the three models (batch size = pipeline stages). Using this configuration, CENT achieves a geomean of 2.3× higher end-to-end throughput across three models. CENT demonstrates 1.2× speedup on Llama2-70B because this model applies the grouped-query attention technique [3], improving the operational intensity of the attention layers. Figure 13(c) shows that CENT processes 5.2× higher tokens per dollar than GPU, which attributes to CENT's higher throughput and 2.5× cheaper TCO (Table 4).

Figure 13(b) compares throughput in the prefill and decoding stages. GPU achieves 2.5× higher throughput in the compute-intensive prefill stage than CENT due to GPU's 2.0× higher peak compute throughput. Conversely, CENT outperforms GPU in the memory-intensive decoding stage by 2.5× due to PIM's higher internal memory bandwidth. Notably, the prefill stage accounts for only 2% of the total GPU end-to-end processing time, so the overall LLM inference throughput closely aligns with that of the decoding stage.

CENT performs better than GPU in long context scenarios. The results in Figure 13 use a 4K context window. However, state-of-the-art LLMs support longer contexts, ranging from 128K to 1M tokens [29, 81]. As discussed in Section 2, with longer contexts, the GPU system saturates at smaller batch sizes, from batch=128 at 4K context to batch=16 at 32K context. On Llama2-70B, CENT achieves higher speedup than GPUs as context length increases, attaining up to 3.3× speedup in decoding throughput for a context length of 32K, as shown in Figure 14(a).

CENT has lower query latency than GPU at similar throughput. Figure 14(b) illustrates our QoS comparison on Llama2-70B. These results are collected with different batch sizes on GPUs and different TP/PP mapping strategies on CENT. CENT provides 3.4-7.6× lower query latency while achieving similar throughput to the baseline GPU.

Latency Breakdown. Figure 14(c) shows CENT's latency breakdown with different TP/PP mapping strategies. PIM latency always dominates because most of the operations are mapped to PIM channels. As TP increases (from top to bottom), PIM latency reduces. This is because more PIM channels are allocated to a single transformer block. Yet, CXL communication latency increases with higher TP, because distributing a transformer block across more CXL devices necessitates more broadcast and gather transactions. Figure 14(d) depicts the latency comparison between CENT and GPU at maximum supported batch sizes. Compared to GPU, CENT shows 1.4× higher latency in the prefill stage and 1.7-2.0× lower latency in the decoding stage. Decoding latency dominates the end-to-end latency.

7.2 Power and Energy Consumption Analysis

We developed an *activity-based* power model for CENT. When deploying the Llama2-70B model on 32 CXL devices with the pipeline parallel model mapping, 27 devices are used. Among 80 transformer blocks (80 pipeline stages), 3 of them are mapped to each device, resulting in an average power of 32.4W per device. PIM operations and activation/precharge commands consume 54.5% and 30.2% of power, respectively.

Similarly, we used `nvidia-smi` to measure GPU power during the prefill and decoding stages in 100ms intervals. Figure 15(a) illustrates the average power consumption of CENT versus Nvidia A100 80GB GPUs. *One* A100 GPU consumes ≈ 8× higher power than *one* CENT device. Modern GPUs consume significantly higher power as they support

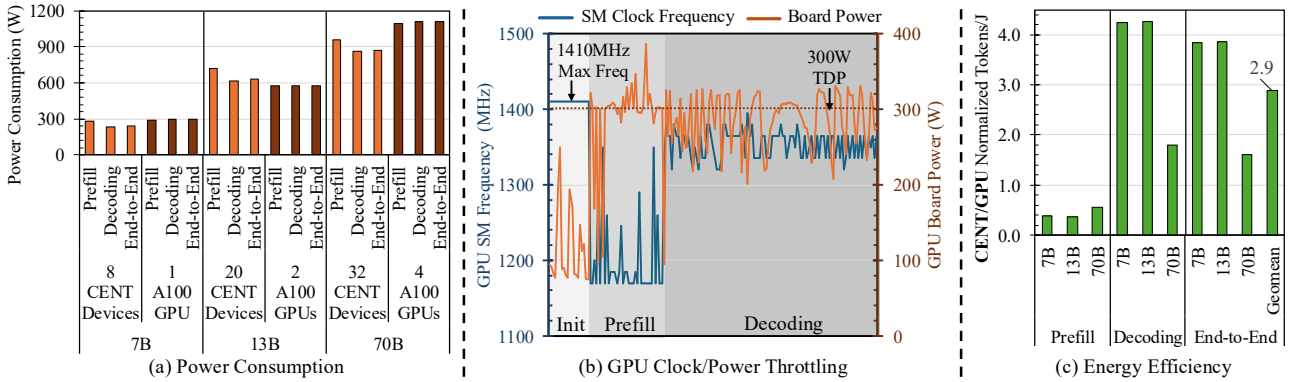


Figure 15. (a) Power consumption of CENT and GPU (b) GPU SM frequency and board power, and (c) energy efficiency (Tokens per Joule) of CENT and GPU using the maximum batch size, 512 prefill tokens and 3584 decoding tokens.

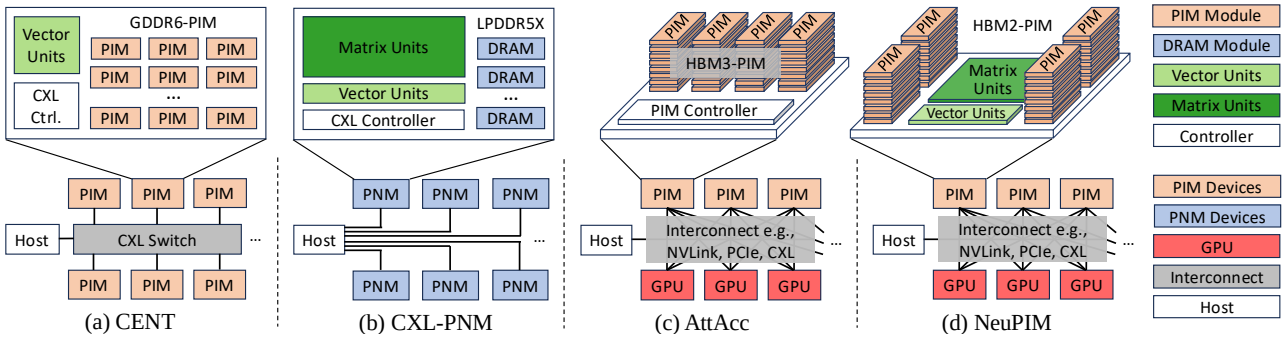


Figure 16. (a) CENT employs vector units near PIM modules and utilizes a CXL switch to interconnect PIM devices with novel CXL communication primitives. (b) CXL-PNM [88] applies a processing-near-memory solution *without* integrating compute logic into DRAM chips. (c-d) AttAcc [86] and NeuPIM [37] are heterogeneous systems comprising GPUs and PIM devices.

general-purpose PTX ISA [77], a large number of Streaming Multiprocessors (108 SMs in A100), multithreading with fast context switching, and a multi-level cache hierarchy (≈ 60 MB in A100 [74]). In contrast, CENT is a custom architecture with minimal silicon used for near-bank compute units.

GPUs operate near their thermal design power (TDP) of 300W [74] during both the prefill and decoding stages when processing a large batch size of 128 queries. Figure 15(b) illustrates this by showing the GPU's SM clock frequency and board power consumption for the Llama2-7B model. During vLLM [54] initialization, the clock frequency is maximized at 1410 MHz due to low compute throughput and memory bandwidth utilization. In the prefill stage, high SM utilization signals the GPU's power manager to throttle the clock frequency, maintaining power consumption within the TDP. During the decoding stage, reduced SM utilization allows for an increase in clock frequency. A higher clock rate and memory bandwidth usage keep power near the TDP.

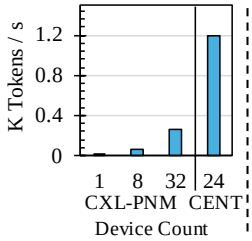
Figure 15(c) shows that CENT processes 2.9× more *tokens per Joule* than GPU, on average. In the compute-bound prefill stage, GPU is 2.4× more energy efficient, as it achieves efficient data reuse in the on-chip SRAM. In the memory-bound decoding stage, CENT achieves 3.2× higher energy efficiency, while GPU cannot efficiently reuse data in the SRAM because of the low operational intensity. Our evaluation shows that CENT consumes 0.6 pJ/bit on `MAC_ABK` operations, making it 6.6× more energy efficient than even *only* the HBM2 memory read accesses of GPU, which consumes 3.97 pJ/bit [78].

7.3 CENT versus PIM/PNM Baselines

We compare CENT with the state-of-the-art CXL-PNM [88] and heterogeneous GPU-PIM baselines [37, 86]. Figure 16 provides an architectural overview of these systems.

CENT versus CXL-PNM. Figure 16(b) shows that CXL-PNM [51, 88] is a processing-near-memory (PNM) platform that leverages a CXL controller to manage eight LPDDR5X packages within a single device. The CXL controller deploys matrix and vector units to perform computations *near* commodity LPDDR5X chips. In contrast, Figure 16(a) depicts CENT, which utilizes processing-in-memory (PIM) technology to place compute logic adjacent to DRAM banks *within* DRAM chips. Figure 17(b) shows that compared to CXL-PNM, CENT provides significantly higher compute throughput (TFLOPs) and memory bandwidth (TB/s), at the cost

of less memory capacity (GB). Figure 17(a) illustrates that CENT's higher compute and memory bandwidth results in 4.5× higher throughput than CXL-PNM, at the maximum supported batch sizes for each system.



(a) CENT and CXL-PNM

System	CXL-PNM			CENT
Device Count	1	8	32	24
TFLOPS	8.2	65.6	262	456
Mem BW (TB/s)	1.1	8.8	35.2	384
Mem Capacity	512 GB	4 TB	16 TB	384 GB

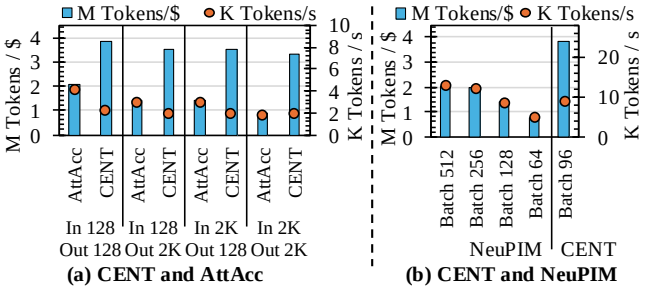
(b) System Configurations

Figure 17. CENT and CXL-PNM baseline comparison on OPT-66B [114] with prefill=64 and decoding=1024.

CENT versus GPU-PIM. AttAcc [86] and NeuPIM [37] are heterogeneous systems consisting of GPUs and PIM devices as shown in Figure 16(c) and (d). The AttAcc system consists of 8 A100 GPUs with HBM3 memory [73] and 8 HBM-PIM devices. Each HBM-PIM device consumes 116W and has a memory capacity of 80GB. The NeuPIM device integrates a TPUv4-like NPU [45] architecture near PIM modules and extends PIM with dual row buffers, enabling concurrent PIM-NPU memory access. The evaluated NeuPIM platform comprises 8 A100 GPUs and 8 NeuPIM devices.

Distinct from these systems, CENT introduces a GPU-free inference server, providing an alternative cost-effective solution and eliminating the need for expensive GPUs. In GPU-PIM systems, the prefill stage is mapped to GPUs while the remaining computation is mapped to the PIM subsystem. CENT does *not* employ GPUs for the prefill stage for various reasons. *First*, end-to-end LLM inference performance is primarily constrained by the decoding phase rather than the prefill phase; only 2% of the total GPU's inference time is taken by the prefill stage across Llama2 models, on average (Section 7.1). *Second*, CENT's compute throughput is not much worse than GPU (≈49%, Table 4). *Third*, using expensive GPUs solely to support the prefill stage is a costly option. Using the methodology from Section 6, we find that the TCO of AttAcc and NeuPIM is 3.5× and 2.6× higher than CENT, respectively. The cost of HBM-PIM is estimated at 10× the price of HBM [98], while the NPU cost is modeled based on die, 2.5D packaging, and NRE costs [45, 49, 85].

Figure 18 shows the performance of CENT versus AttAcc and NeuPIM. For a power-neutral evaluation, we assume 12 CENT devices per GPU-PIM node. Across different sequence lengths and batch sizes, the blue bars show that CENT processes 1.8-3.7× and 1.8-5.3× more tokens per dollar than AttAcc and NeuPIM systems, respectively. The orange dots show that CENT's raw throughput (Tokens/s) is 0.5-1.1× and 0.7-2.1× the throughput of AttAcc and NeuPIM, respectively. In scenarios with short sequence lengths, query batching



(a) CENT and AttAcc (b) CENT and NeuPIM

Figure 18. CENT versus GPU-PIM (a) CENT and AttAcc systems are evaluated on the GPT3-175B model across various input and output sizes, tested at the maximum supported batch sizes. (b) The CENT and NeuPIM systems are evaluated on GPT3-175B with data-parallel mapping (DP=4) and pipeline-parallel mapping (PP=4), respectively, using the ShareGPT dataset [105]. NeuPIM uses different batch sizes while CENT uses the maximum supported batch size 96.

enhances operational intensity in FC layers, improving performance on GPUs (or NPUs) with more TFLOPs. However, in cases with long sequence lengths that limit batch sizes, CENT maintains higher raw throughput than the GPU-PIM baselines. Latest LLM models typically support 128K context windows [81]. With these extended context lengths, we expect CENT to provide even higher performance.

7.4 Design Space Exploration

CENT can interconnect a flexible number of CXL devices, allowing for scalable system configurations. Figure 19 shows the scalability of CENT on Llama2-70B from 16 to 128 devices, with throughput increasing from 0.68 K tokens/s to 5.7 K tokens/s. We start with pipeline-parallel (PP) mapping and then apply various levels of data-parallel (DP) mapping to further boost the throughput as the CENT system scales up. As the number of devices increases, the throughput reaches intermittent plateaus at certain points. This is due to the inefficiency of distributing transformer blocks across CXL devices. For example, 80 transformer blocks in the Llama2-70B model can be allocated to 40 devices, with two blocks per device. Expanding from 40 to 44 devices results in a distribution of 1.8 blocks per device. Yet, dividing a single block across multiple CXL devices introduces substantial inter-device communication overhead, ultimately reducing performance. To mitigate this, we maintain the same block distribution with 44 devices as with 40, leaving the remaining 4 devices idle.

The scalability of CXL devices is constrained by two primary factors: (1) The number of lanes and ports provided by a CXL switch. For example, a commercial PCIe 5.0 switch can accommodate up to 144 lanes and 72 ports [8]. (2) The maximum power supply available for the server, such as the DGX A100's peak input power of 6.5 kW [73]. Due to these constraints, the CENT system with a single switch can

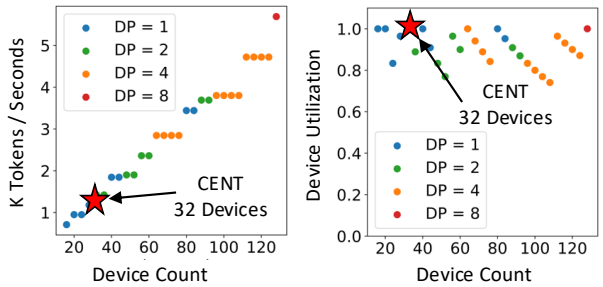


Figure 19. CENT scalability study on Llama2-70B.

support up to 64 devices per server. A larger number of devices can be driven by multi-socket CPUs or a memory pool implementation facilitated by two levels of CXL switches.

7.5 Generality

LLMs exhibit similar architectures but differ in their specific implementations of activation functions and positional encodings. CENT is designed to support a variety of activation functions, including GeLU [36], Swish [95], and their GLU variants [99]. This versatility is achieved by decomposing these functions into fundamental non-linear operations, such as `sigmoid` and `tanh`, which are supported through lookup tables, as well as through basic PIM and RISC-V operations. Moreover, CENT is capable of accommodating different types of positional embeddings, including both absolute [92] and relative [102] implementations. The integration of general-purpose RISC-V cores within the CENT system opens up possibilities for further enhancements and optimizations of LLMs in the future.

8 Related Work

Various ML accelerators and HW/SW co-designs have recently been proposed [11, 53, 62]. CXL memory expansion techniques are also widely explored [2, 5, 30, 31, 44, 103]. Sections 1 and 2 already discuss PIM and PNM related works.

Transformer Accelerators. A variety of transformer accelerators [34, 66, 93, 94] have been developed to enhance this prevalent ML architecture. TransPIM [117] accelerates inference of transformer encoders like BERT [16] by reducing data loading time with an efficient token-based dataflow. However, decoder-only LLM's inference tasks present a unique challenge due to their lower operational intensities, which have been less investigated. Approaches like Sprint [112], OliVe [32], FABNet [23], and SpAtten [109] employ quantization, approximation, and pruning strategies, respectively, aimed at reducing computations within the transformer blocks, which are orthogonal to CENT.

CXL-Based NDP Accelerators. Samsung's CXL-PNM platform [51, 88] integrates an LLM inference accelerator in the CXL controller. CENT also integrates PIM memory chips with PUs adjacent to DRAM banks, providing both higher internal memory bandwidth and compute throughput than CXL-PNM. Beacon [39] explores near-data processing in both DIMMs and CXL switches, with customized processing units for accelerating genome sequencing analysis.

9 Conclusion

Given the challenges posed by the low operational intensity and substantial memory capacity requirements of decoder-only LLMs, we introduce CENT, utilizing PIM technology to facilitate the high internal memory bandwidth and CXL memory expansion to ensure ample memory capacity. When compared to GPU baselines with the maximum supported batch sizes, CENT achieves 2.3× higher throughput and consumes 2.3× less energy. CENT also enables lower TCO and generates 5.2× more tokens per dollar than GPUs.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. This work was generously supported by NSF CAREER-1652294, NSF-1908601 and Intel gift awards. SAFARI authors acknowledge support from the Semiconductor Research Corporation, ETH Future Computing Laboratory (EFCL), AI Chip Center for Emerging Smart Systems Limited (ACCESS), and the European Union's Horizon Programme for research and innovation under Grant Agreement No. 101047160.

A Artifact Appendix

A.1 Abstract

This document provides a concise guide for reproducing the main performance, power, cost efficiency, and energy efficiency results of this paper in Figures 12, 13, 14, and 15. The instructions cover the steps required to clone the GitHub repository, build the simulator, set up the necessary Python packages, execute the end-to-end simulation, process results, and generate figures. The trace generator, performance simulator, power model, automation scripts, expected results, and detailed instructions are available in our GitHub repository.

A.2 Artifact check-list (meta-information)

- **Program:** C++ and Python.
- **Compilation:** `g++-11/12/13` or `clang++-15`.
- **Software:** `pandas`, `matplotlib`, `torch`, and `scipy` Python packages.
- **Model:** Llama2 7B, 13B, and 70B [106].
- **Metrics:** latency, throughput (tokens/S), cost efficiency (tokens/$), energy efficiency (tokens/J), and power.
- **Output:** CSV and PDF files corresponding to Figures 12-15.
- **Experiments:** PIM trace generation and simulation, and CENT power modeling.
- **How much disk space is required?:** Approximately 100GB.
- **How much time is needed?:** Approximately 24 hours on a desktop and 8 12 hours on a server.
- **Publicly available?:** Available on GitHub and Zenodo.
- **Code licenses:** MIT License.
- **Work automation?:** Automated by a few scripts.

A.3 Description

This artifact provides the necessary components to reproduce the main results presented in Figures 12, 13, 14, and 15. It includes a trace generator, AiM simulator, power model, figure generator, and automation script. While these figures incorporate simulation results from CENT, they also rely on a baseline GPU system featuring four Nvidia A100 80GB GPUs, as detailed in Table 4. Due to the high cost associated with these servers, only the expected results for the GPU baseline system are provided in the data directory.

A.3.1 How to access. Clone the artifact from our GitHub repository using the following command. Please do not forget the `--recursive` flag to ensure that the AiM simulator is also cloned:

```
git clone --recursive https://github.com/Yufeng98/
    CENT.git
```

A.3.2 Software dependencies. AiM simulator requires `g++-11/12/13` or `clang++-15` for compilation. The Python infrastructure requires `pandas`, `matplotlib`, `torch`, and `scipy` packages.

A.3.3 Models. Section 6 shows that we evaluate three Llama2 models [106]. The model architecture and its PIM mapping are implemented in the `cent_simulation/Llama.py` script. The model weights are required only for the functional simulation of the PIM infrastructure. While the functional simulator is available in our GitHub repository, the performance simulator and power model described in this appendix do not model real values, as this does not impact the main results. Consequently, the model weights and parameters are not required for this appendix.

A.4 Installation

Building AiM Simulator. To build the simulator, use the following script:

```
cd CENT/aim_simulator/
mkdir build && cd build && cmake ..
make -j4
```

Setting up Python Packages. Install the aforementioned Python packages. You can use the following script to create a conda environment:

```
cd CENT/
conda create -n cent python=3.10 -y
conda activate cent
pip install -r requirements.txt
```

A.5 Experiment workflow

We provide scripts to facilitate the end-to-end reproduction of the results. The following steps outline the process.

Generate and Simulate the Traces. This step generates and simulates all required PIM traces. It also processes the simulation logs, calculates individual latencies, and utilizes the CENT power model to determine energy consumption and average power. Upon completion, the generated trace and simulation log files will be stored in the `trace` directory, while the processed latency and power results can be found in `cent_simulation/simulation_results.csv`.

```
cd CENT/
bash remove_old_results.sh

cd cent_simulation/
bash simulation.sh [NUM_THREADS] [SEQ_GAP]
```

Note: The argument `[NUM_THREADS]` should be set according to the number of available parallel threads on your processor. For instance, 8 threads are recommended for desktop processors, while server processors can utilize 96 threads.

The argument `[SEQ_GAP]` determines the gap between each simulated token. Setting this value to one simulates every token sequentially, requiring approximately 100GB of disk space and taking around 24 hours on a processor with 8 threads or 12 hours on a processor with 96 threads. To improve disk usage and reduce simulation time, the `[SEQ_GAP]` argument can be set to a larger value, such as 128. This configuration simulates one out of every 128 tokens, processing token IDs of 128, 256, 384, and so on up to 4096.

Process the Results. This step processes the simulation results and computes the latency, throughput, power, and energy for the prefill, decoding, and end-to-end phases. After processing the results, this script stores them in this file: `cent_simulation/processed_results.csv`.

```
cd CENT/cent_simulation/
bash process_results.sh
```

Generate Figures. The following script generates Figures 12-15. This process utilizes the baseline GPU results, available in the data directory, along with the processed results. It computes the normalized results and generates both a PDF file containing the figures and a CSV file with the corresponding numerical data.

```
cd CENT/
bash generate_figures.sh
```

A.6 Evaluation and expected results

The normalized results and the figures will be located in the `figure_source_data` and `figures` directories. The expected results can be found in Figures 12- 15 or in the generated CSV and PDF files on our GitHub repository. Figures in the paper are generated using Microsoft Excel. To visualize the figures in the paper's format, copy the normalized data from the CSV files to the `Data` sheet of the provided Figures.xlsx. Figures will be generated in the `Figures` sheet.

References

[1] Azure pricing calculator. URL: https://azure.microsoft.com/en-us/pricing/calculator/.

[2] Minseon Ahn, Andrew Chang, Donghun Lee, Jongmin Gim, Jungmin Kim, Jaemin Jung, Oliver Rebholz, Vincent Pham, Krishna Malladi, and Yang Seok Ki. Enabling cxl memory expansion for in-memory database management systems. In *Proceedings of the 18th International Workshop on Data Management on New Hardware*, pages 1–5, 2022.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL: https://arxiv.org/abs/2305.13245, arXiv:2305.13245.

[4] Anthropic. Introducing the next generation of claude. URL: https://www.anthropic.com/news/claude-3-family.

[5] Moiz Arif, Kevin Assogba, M Mustafa Rafique, and Sudharshan Vazhkudai. Exploiting CXL-based memory for distributed deep learning. In *Proceedings of the 51st International Conference on Parallel Processing*, pages 1–11, 2022.

[6] Thomas Atta-fosu. Llama 2 70b: An mlperf inference benchmark for large language models. URL: https://mlcommons.org/2024/03/mlperf-llama2-70b/.

[7] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, Wesam Manassra, Prafulla Dhariwal, Casey Chu, Yunxin Jiao, and Aditya Ramesh. Improving image generation with better captions. *Computer Science.*, 2(3):8, 2023. URL: https://cdn.openai.com/papers/dall-e-3.pdf.

[8] Broadcom. 144-lane, 72-port, pci express gen 5.0 pex89144 expressfabric platform. URL: https://www.broadcom.com/products/pcie-switches-bridges/expressfabric/gen5/pex89144.

[9] Christopher Celio, Krste Asanovic, and David Patterson. The berkeley out-of-order machine (boom): An open-source industry-competitive, synthesizable, parameterized risc-v processor. URL: https://riscv.org/wp-content/uploads/2016/01/Wed1345-RISCV-Workshop-3-BOOM.pdf.

[10] Christopher Celio, Pi-Feng Chiu, Borivoje Nikolic, David A. Patterson, and Krste Asanović. BOOM v2: an open-source out-of-order RISC-V core. Technical Report UCB/EECS-2017-157, EECS Department, University of California, Berkeley, Sep 2017. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2017/EECS-2017-157.html.

[11] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits*, 52(1):127–138, 2016.

[12] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307*, 2023.

[13] Fabrice Devaux. The true processing in memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–24. IEEE Computer Society, 2019.

[14] Alexandar Devic, Siddhartha Balakrishna Rai, Anand Sivasubramaniam, Ameen Akel, Sean Eilert, and Justin Eno. To pim or not for emerging general purpose processing in ddr memory systems. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 231–244, 2022.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. URL: http://arxiv.org/abs/1810.04805, arXiv:1810.04805.

[17] dramexchange. Dram spot price. URL: https://www.dramexchange.com/.

[18] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[19] Afzal Ahmad Dylan Patel. The inference cost of search disruption – large language model cost analysis. URL: https://www.semianalysis.com/p/the-inference-cost-of-search-disruption.

[20] ebay. Nvidia tesla a100 80gb gpu sxm4 deep learning computing graphics card oem. URL: https://www.ebay.com/itm/126596600113?chn=ps&mkevt=1&mkcid=28&srsltid=AfmBOop8-DCL9WiHC15MU05ZikXFveIxl95uEuqd55d5LHBrMjRXNMiwSTg.

[21] Mouser Electronics. Pci interface ic. URL: https://www.mouser.com/c/semiconductors/interface-ics/pci-interface-ic/.

[22] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017. URL: https://arxiv.org/abs/1702.03118, arXiv:1702.03118.

[23] Hongxiang Fan, Thomas Chau, Stylianos I Venieris, Royson Lee, Alexandros Kouris, Wayne Luk, Nicholas D Lane, and Mohamed S Abdelfattah. Adaptable Butterfly Accelerator for Attention-based NNs via Hardware and Algorithm Co-design. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 599–615. IEEE, 2022.

[24] Amin Farmahini-Farahani, Jung Ho Ahn, Katherine Morrow, and Nam Sung Kim. NDA: Near-DRAM acceleration architecture leveraging commodity DRAM devices and standard memory modules. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, pages 283–295. IEEE, 2015.

[25] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam Sapek, Gabriel Weisz, Lisa Woods, Sitaram Lanka, Steven K. Reinhardt, Adrian M. Caulfield, Eric S. Chung, and Doug Burger. A configurable cloud-scale DNN processor for real-time AI. In 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA). 2018. URL: https://doi.org/10.1109/isca.

[26] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. Sparsep: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 6(1):1–49, 2022.

[27] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system. *IEEE Access*, 10:52565–52608, 2022.

[28] Juan Gómez-Luna, Yuxin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F Oliveira, Gagandeep Singh, and Onur Mutlu. Evaluating machine learningworkloads on memory-centric computing systems. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 35–49. IEEE, 2023.

[29] Google. Our next-generation model: Gemini 1.5. URL: https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/.

[30] Donghyun Gouk, Miryeong Kwon, Hanyeoreum Bae, Sangwon Lee, and Myoungsoo Jung. Memory pooling with cxl. *IEEE Micro*, 43(2):48–57, 2023.

[31] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct access, High-Performance memory disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 287–294, 2022.

[32] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. OliVe: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization. In *Proceedings of the 50th Annual International*

Symposium on Computer Architecture, pages 1–15, 2023.

[33] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[34] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, Soosung Kim, Hyunji Choi, Sung Jun Jung, and Jae W Lee. ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 692–705. IEEE, 2021.

[35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[36] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[37] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 722–737, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3620666.3651380.

[38] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. *GPipe: efficient training of giant neural networks using pipeline parallelism*. Curran Associates Inc., Red Hook, NY, USA, 2019.

[39] Wenqin Huangfu, Krishna T Malladi, Andrew Chang, and Yuan Xie. BEACON: Scalable Near-Data-Processing Accelerators for Genome Analysis near Memory Pool with the CXL Support. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 727–743. IEEE, 2022.

[40] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. Floatpim: In-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 802–815, 2019.

[41] Mohsen Imani, Saransh Gupta, and Tajana Rosing. Ultra-efficient processing in-memory for data intensive applications. In *Proceedings of the 54th Annual Design Automation Conference 2017*, pages 1–6, 2017.

[42] Intel. Intel xeon gold 6430 processor, 60m cache, 2.10 ghz. URL: https://www.intel.com/content/www/us/en/products/sku/231737/intel-xeon-gold-6430-processor-60m-cache-2-10-ghz/specifications.html.

[43] Intel. Intel® xeon® gold 6430 processor. URL: https://www.intel.com/content/www/us/en/products/sku/231737/intel-xeon-gold-6430-processor-60m-cache-2-10-ghz/specifications.html.

[44] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 585–600, 2023.

[45] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3579371.3589350.

[46] Sanjay Kariyappa, Hsinyu Tsai, Katie Spoon, Stefano Ambrogio, Pritish Narayanan, Charles Mackin, An Chen, Moinuddin Qureshi, and Geoffrey W Burr. Noise-resilient DNN: Tolerating noise in PCM-based AI accelerators via noise-aware training. *IEEE Transactions on Electron Devices*, 68(9):4356–4362, 2021.

[47] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller, Jochen Kuhn, and Gjergji Kasneci. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.

[48] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, and Xuan Zhang. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 790–803. IEEE, 2020.

[49] Moein Khazraee, Lu Zhang, Luis Vega, and Michael Bedford Taylor. Moonwalk: Nre optimization in asic clouds. *ACM SIGARCH Computer Architecture News*, 45(1):511–526, 2017.

[50] Jin Hyun Kim, Shin-Haeng Kang, Sukhan Lee, Hyeonsu Kim, Yuhwan Ro, Seungwon Lee, David Wang, Jihyun Choi, Jinin So, YeonGon Cho, Kyomin Sohn, and Nam Sung Kim. Aquabolt-XL HBM2-PIM, LPDDR5-PIM with in-memory processing, and AXDIMM with acceleration buffer. *IEEE Micro*, 42(3):20–30, 2022.

[51] Jin Hyun Kim, Yuhwan Ro, Jinin So, Sukhan Lee, Shin-haeng Kang, YeonGon Cho, Hyeonsu Kim, Byeongho Kim, Kyungsoo Kim, Sangsoo Park, Jin-Seong Kim, Sanghoon Cha, Won-Jo Lee, Jin Jung, Jong-Geon Lee, Jieun Lee, JoonHo Song, Seungwon Lee, Jeonghyeon Cho, Jaehoon Yu, and Kyomin Sohn. Samsung PIM/PNM for Transfmer Based AI: Energy Efficiency on PIM/PNM Cluster. In *2023 IEEE Hot Chips 35 Symposium (HCS)*, pages 1–31. IEEE Computer Society, 2023.

[52] Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gi-Moon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Vladimir Kornijcuk, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jaewook Lee, Donguc Ko, Younggun Jun, Ilwoong Kim, Choungki Song, Ilkon Kim, Chanwook Park, Seho Kim, Chunseok Jeong, Euicheol Lim, Dongkyun Kim, Jieun Jang, Il Park, Junhyun Chun, and Joohwan Cho. A 1ynm 1.25v 8gb 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep learning application. *IEEE Journal of Solid-State Circuits*, 58(1):291–302, 2023. doi:10.1109/JSSC.2022.3200718.

[53] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices*, 53(2):461–475, 2018.

[54] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[55] Yongkee Kwon, Guhyun Kim, Nahsung Kim, Woojae Shin, Jongsoon Won, Hyunha Joo, Haerang Choi, Byeongju An, Gyeongcheol Shin, Dayeon Yun, Jeongbin Kim, Changhyun Kim, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyeongdeok Lee, Seungyeong Park, Wonjun Lee, Seongju Lee, Kyuyoung Kim, Daehan Kwon, Chunseok Jeong, John Kim, Euicheol Lim, and Junhyun Chun. Memory-centric computing with sk hynix's domain-specific memory. In *2023 IEEE Hot Chips 35 Symposium (HCS)*, pages 1–26, 2023. doi:

10.1109/HCS59251.2023.10254717.

[56] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, and Junhyun Chun. System architecture and software stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS)*, pages 1–25. IEEE, 2022.

[57] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, Youngmin Cho, Jin Guk Kim, Jongyoon Choi, Hyun-Sung Shin, Jin Kim, BengSeng Phuah, HyoungMin Kim, Myeong Jun Song, Ahn Choi, Daeho Kim, SooYoung Kim, Eun-Bong Kim, David Wang, Shinhaeng Kang, Yuhwan Ro, Seungwoo Seo, JoonHo Song, Jaeyoun Youn, Kyomin Sohn, and Nam Sung Kim. 25.4 a 20nm 6gb function-in-memory DRAM, based on HBM2 with a 1.2 tflops programmable computing unit using bank-level parallelism, for machine learning applications. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 64, pages 350–352. IEEE, 2021.

[58] Donghun Lee, Jinin So, Minseon Ahn, Jong-Geon Lee, Jungmin Kim, Jeonghyeon Cho, Rebholz Oliver, Vishnu Charan Thummala, Ravi shankar JV, Sachin Suresh Upadhya, Donghun Lee, Jinin So, Minseon Ahn, Jong-Geon Lee, Jungmin Kim, Jeonghyeon Cho, Rebholz Oliver, Vishnu Charan Thummala, Ravi shankar JV, Sachin Suresh Upadhya, Mohammed Ibrahim Khan, and Jin Hyun Kim. Improving in-memory database operations with acceleration DIMM (AxDIMM). In *Proceedings of the 18th International Workshop on Data Management on New Hardware*, pages 1–9, 2022.

[59] Melvin Lee. Using machine learning to increase yield and lower packaging costs. URL: https://semiengineering.com/using-machine-learning-to-increase-yield-and-lower-packaging-costs/.

[60] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Kornijcuk Vladimir, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Jaewook Lee, Donguc Ko, Younggun Jun, Keewon Cho, Ilwoong Kim, Choungki Song, Chunseok Jeong, Daehan Kwon, Jieun Jang, Il Park, Junhyun Chun, and Joohwan Cho. A 1ynm 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3. IEEE, 2022.

[61] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 574–587, 2023.

[62] Youjie Li, Iou-Jen Liu, Yifan Yuan, Deming Chen, Alexander Schwing, and Jian Huang. Accelerating distributed reinforcement learning with in-switch computing. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 279–291, 2019.

[63] Compute Express Link™. Specification. URL: https://www.computeexpresslink.org/.

[64] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[65] Liu Liu, Jilan Lin, Zheng Qu, Yufei Ding, and Yuan Xie. Enmc: Extreme near-memory classification via approximate screening. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 1309–1322, 2021.

[66] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 977–991, 2021.

[67] Haocong Luo, Yahya Can Tuğrul, F Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *arXiv preprint arXiv:2308.11030*, 2023.

[68] Siming Ma, David Brooks, and Gu-Yeon Wei. A Binary-activation, Multi-level Weight RNN and Training Algorithm for ADC-/DAC-free and Noise-resilient Processing-in-memory Inference with eNVM. *IEEE Transactions on Emerging Topics in Computing*, 2023.

[69] Micron. Dram power calculator. URL: https://www.micron.com/support/tools-and-utilities/power-calc.

[70] Jowi Morales. Nvidia shipped 3.76m data center gpus in 2023. URL: https://www.tomshardware.com/tech-industry/nvidia-shipped-376m-data-center-gpus-in-2023-dominates-business-with-98-revenue-share.

[71] August Ning, Georgios Tziantzioulis, and David Wentzlaff. Supply chain aware computer architecture. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[72] Dimin Niu, Shuangchen Li, Yuhao Wang, Wei Han, Zhe Zhang, Yijin Guan, Tianchan Guan, Fei Sun, Fei Xue, Lide Duan, Yuanwei Fang, Hongzhong Zheng, Xiping Jiang, Song Wang, Fengguo Zuo, Yubing Wang, Bing Yu, Qiwei Ren, and Yuan Xie. 184QPS/W 64Mb/mm 2 3D logic-to-DRAM hybrid bonding with process-near-memory engine for recommendation system. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3. IEEE, 2022.

[73] NVIDIA. Introduction to the nvidia dgx a100 system. URL: https://docs.nvidia.com/dgx/dgxa100-user-guide/introduction-to-dgxa100.html#power-specifications.

[74] NVIDIA. Nvidia a100 tensor core gpu. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf.

[75] Nvidia. Nvidia hgx a100, the most powerful end-to-end ai supercomputing platform. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/HGX/a100-80gb-hgx-a100-datasheet-us-nvidia-1485640-r6-web.pdf.

[76] Nvidia. Nvlink and nvlink switch. URL: https://www.nvidia.com/en-us/data-center/nvlink/.

[77] NVIDIA. Ptx isa. URL: https://docs.nvidia.com/cuda/pdf/ptx_isa_8.5.pdf.

[78] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W Keckler, and William J Dally. Fine-grained dram: Energy-efficient dram for extreme bandwidth systems. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 41–54, 2017.

[79] U.S. Bureau of Labor Statistics. Average energy prices for the united states, regions, census divisions, and selected metropolitan areas. URL: https://www.bls.gov/regions/midwest/data/averageenergyprices_selectedareas_table.htm.

[80] Geraldo F Oliveira, Juan Gómez-Luna, Saugata Ghose, Amirali Boroumand, and Onur Mutlu. Accelerating neural network inference with processing-in-dram: From the edge to the cloud. *IEEE Micro*, 42(6):25–38, 2022.

[81] OpenAI. Gpt-4 turbo and gpt-4. URL: https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4.

[82] OpenAI. Learning to reason with llms. URL: https://openai.com/index/learning-to-reason-with-llms/.

[83] OpenAI. Video generation models as world simulators. URL: https://openai.com/research/video-generation-models-as-world-simulators.

[84] OpenAI. GPT-4 Technical Report, 2023. arXiv:2303.08774.

[85] Chet Palesko, Amy Palesko, and E Jan Vardaman. Cost and yield analysis of multi-die packaging using 2.5 d technology compared to fan-out wafer level packaging. In *Proceedings of the 5th Electronics System-integration Technology Conference (ESTC)*, pages 1–5. IEEE, 2014.

[86] Jaehyun Park, Jaewan Choi, Kwanhee Kyung, Michael Jaemin Kim, Yongsuk Kwon, Nam Sung Kim, and Jung Ho Ahn. Attacc! unleashing the power of pim for batched transformer-based generative model inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 103–119, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3620665.3640422.

[87] Jaehyun Park, Byeongho Kim, Sungmin Yun, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 268–281, 2021.

[88] Sang-Soo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyoungwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, Jinhyun Kim, Jieun Lee, YeonGon Cho, Yongmin Tai, Jeonghyeon Cho, Hoyoung Song, Jung Ho Ahn, and Nam Sung Kim. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 970–982. IEEE, 2024.

[89] Dylan Patel. Nvidia ada lovelace leaked specifications, die sizes, architecture, cost, and performance analysis. URL: https://www.semianalysis.com/p/nvidia-ada-lovelace-leaked-specifications.

[90] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[91] Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. Movie gen: A cast of media foundation models. *arXiv preprint arXiv:2410.13720*, 2024.

[92] Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

[93] Yubin Qin, Yang Wang, Dazheng Deng, Zhiren Zhao, Xiaolong Yang, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. FACT: FFN-Attention Co-optimized Transformer Architecture with Eager Correlation Prediction. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–14, 2023.

[94] Zheng Qu, Liu Liu, Fengbin Tu, Zhaodong Chen, Yufei Ding, and Yuan Xie. Dota: detect and omit weak attentions for scalable transformer acceleration. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 14–26, 2022.

[95] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[96] Elaheh Sadredini, Reza Rahimi, Marzieh Lenjani, Mircea Stan, and Kevin Skadron. Impala: Algorithm/architecture co-design for in-memory multi-stride pattern matching. In *2020 IEEE international symposium on high performance computer architecture (HPCA)*, pages 86–98. IEEE, 2020.

[97] Samsung. 8gb gddr6 sgram c-die. URL: https://datasheet.lcsc.com/lcsc/2204251615_Samsung-K4Z80325BC-HC14_C2920181.pdf.

[98] Kiwoom Securities. Generative ai winds in memory semiconductors, total demand for server drams is declining. URL: https://www.businesspost.co.kr/BP?command=article_view&num=316574.

[99] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[100] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[101] Aaron Stillmaker and Bevan Baas. Scaling equations for the accurate prediction of cmos device performance from 180nm to 7nm. *Integration*, 58:74–81, 2017. URL: https://www.sciencedirect.com/science/article/pii/S0167926017300755, doi:10.1016/j.vlsi.2017.02.002.

[102] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[103] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Ipoom Jeong, Ren Wang, and Nam Sung Kim. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. *arXiv preprint arXiv:2303.15375*, 2023.

[104] Synopsys. Design compiler. concurrent timing, area, power, and test optimization. URL: https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html.

[105] ShareGPT Team. Sharegpt. URL: https://sharegpt.com/.

[106] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[107] UPMEM. Accelerating compute by cramming it into dram memory. URL: https://www.upmem.com/nextplatform-com-2019-10-03-accelerating-compute-by-cramming-it-into-dram/.

[108] Stavros Volos. Memory systems and interconnects for scale-out servers. Technical report, EPFL, 2015.

[109] Hanrui Wang, Zhekai Zhang, and Song Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 97–110. IEEE, 2021.

[110] Wikipedia. Die shot of the tu104 gpu used in rtx 2080 cards. URL: https://en.wikipedia.org/wiki/Turing_(microarchitecture)#/media/File:Nvidia@12nm@Turing@TU104@GeForce_RTX_2080@S_TAIWAN_1841A1_PKYN44.000_TU104-400-A1_DSCx7_poly@5xExt.jpg.

[111] XAI. Open release of grok-1. URL: https://x.ai/blog/grok-os.

[112] Amir Yazdanbakhsh, Ashkan Moradifirouzabadi, Zheng Li, and Mingu Kang. Sparse attention acceleration with synergistic in-memory pruning and on-chip recomputation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 744–762. IEEE, 2022.

[113] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

[114] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[115] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and Intra-Operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, Carlsbad, CA, July 2022. USENIX Association. URL: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin.

[116] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.

[117] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1071–1085. IEEE, 2022.

[118] Farzaneh Zokaee, Fan Chen, Guangyu Sun, and Lei Jiang. Sky-Sorter: A Processing-in-Memory Architecture for Large-Scale Sorting. *IEEE Transactions on Computers*, 72(2):480–493, 2022.

Received 24 June 2024; revised 2 October 2024; accepted 27 January 2025